LAB MANUAL

Draw Basic Graphical Primitives
Mobile Application Development

Complete Source Code with Line-by-Line Explanation

1. Introduction

This lab manual contains the complete source code of the Android project 'Drawsbasicgraphicalprimitives' along with a line-by-line explanation of every Java and XML file. Students should read each section in order to understand the project structure, application workflow, and the role of each line of code.

The app demonstrates basic 2D graphical primitives (circle, rectangle, line, text, oval, arc, and points) using a custom View class and the Android Canvas API.

2. Project Structure

```
Drawsbasicgraphicalprimitives/
├── app/
│   ├── build.gradle.kts                → App build configuration
│   └── src/main/
│       ├── AndroidManifest.xml         → App entry point (OS reads first)
│       ├── java/com/example/drawsbasicgraphicalprimitives/
│       │   ├── MainActivity.java       → Entry Activity (ACTIVE)
│       │   ├── GraphicView.java        → Custom drawing View (ACTIVE)
│       │   ├── FirstFragment.java      → Fragment template (reference)
│       │   └── SecondFragment.java     → Fragment template (reference)
│       └── res/
│           ├── layout/
│           │   ├── activity_main.xml   → Main screen layout (template)
│           │   ├── content_main.xml    → Navigation host layout (template)
│           │   ├── fragment_first.xml  → First fragment UI (template)
│           │   └── fragment_second.xml → Second fragment UI (template)
│           ├── navigation/
│           │   └── nav_graph.xml       → Fragment navigation routes (template)
│           ├── menu/
│           │   └── menu_main.xml       → Options menu (template)
│           ├── values/
│           │   ├── strings.xml         → Text resources
│           │   ├── colors.xml          → Color definitions
│           │   ├── dimens.xml          → Dimension values
│           │   └── themes.xml          → App theme
│           ├── values-v23/
│           │   └── themes.xml          → Theme for API 23+ (edge-to-edge)
│           └── xml/
│               ├── backup_rules.xml    → Backup configuration
│               └── data_extraction_rules.xml → Cloud backup rules
```

File Categories

ACTIVE (used when app runs): MainActivity.java, GraphicView.java, AndroidManifest.xml, themes, strings, colors

TEMPLATE (from Android Studio — study for structure, not shown at runtime): layout files, fragments, navigation, menu

3. Application Workflow

Step 1: User taps app icon
↓
Step 2: Android OS reads AndroidManifest.xml
→ Finds MainActivity as LAUNCHER activity
↓
Step 3: MainActivity.onCreate() executes
→ EdgeToEdge.enable() for full-screen
→ Creates GraphicView object
→ setContentView(graphicView) displays it
→ Applies system bar padding
↓
Step 4: GraphicView is measured and placed on screen
↓
Step 5: GraphicView.onDraw(Canvas) is called
→ All shapes, lines, text are drawn on Canvas
↓
Step 6: Shapes appear on device/emulator screen

4. Java Source Files

File: app/src/main/java/.../MainActivity.java

Purpose: Entry-point Activity. Launched when user opens the app. Creates and displays GraphicView.

Runtime Status: ACTIVE — This file runs when the app starts.

Complete Source Code

```
 1| package com.example.drawsbasicgraphicalprimitives;
 2|
 3| import android.os.Bundle;
 4| import androidx.activity.EdgeToEdge;
 5| import androidx.appcompat.app.AppCompatActivity;
 6| import androidx.core.graphics.Insets;
 7| import androidx.core.view.ViewCompat;
 8| import androidx.core.view.WindowInsetsCompat;
 9|
10| public class MainActivity extends AppCompatActivity {
11|
12|     @Override
13|     protected void onCreate(Bundle savedInstanceState) {
14|         super.onCreate(savedInstanceState);
15|         EdgeToEdge.enable(this);
16|
17|         GraphicView graphicView = new GraphicView(this);
18|         setContentView(graphicView);
19|
20|         ViewCompat.setOnApplyWindowInsetsListener(graphicView, (v, insets) -> {
21|             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
22|             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
23|             return insets;
24|         });
25|     }
26| }
```

Line-by-Line Explanation

Line	Code	Explanation
1	package com.example.drawsbasicgraphicalprimitives;	Declares the package (folder namespace) for this class.
2	(blank line)	Blank line for readability.
3	import android.os.Bundle;	Imports Bundle — holds saved Activity state.
4	import androidx.activity.EdgeToEdge;	Imports EdgeToEdge — enables edge-to-edge display.
5	import androidx.appcompat.app.AppCompatActivity;	Imports AppCompatActivity — backward-compatible Activity base class.
6	import androidx.core.graphics.Insets;	Imports Insets — holds padding values for system bars.
7	import androidx.core.view.ViewCompat;	Imports ViewCompat — compatibility helpers for Views.
8	import androidx.core.view.WindowInsetsCompat;	Imports WindowInsetsCompat — system bar inset information.
9	(blank line)	Blank line separating imports from class.
10	public class MainActivity extends AppCompatActivity {	Defines MainActivity as a public class extending AppCompatActivity.
11	(blank line)	Blank line inside class.
12	@Override	Annotation — this method overrides a parent class method.
13	protected void onCreate(Bundle savedInstanceState) {	onCreate() — first method called when Activity is created.
14	super.onCreate(savedInstanceState);	Calls parent onCreate() — required first step.
15	EdgeToEdge.enable(this);	Enables drawing behind status and navigation bars.
16		Blank line with spaces.
17	GraphicView graphicView = new GraphicView(this);	Creates a new GraphicView object; 'this' passes Activity context.
18	setContentView(graphicView);	Sets GraphicView as the entire screen UI (no XML layout used).
19	(blank line)	Blank line.
20	ViewCompat.setOnApplyWindowInsetsListener(graphicView, (v, insets) -> {	Listens for system bar size changes (status/nav bars).

21	Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());	Gets inset sizes for system bars in pixels.
22	v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);	Adds padding so content is not hidden under bars.
23	return insets;	Returns insets so other listeners can also process them.
24	});	End of lambda listener block.
25	}	End of onCreate() method.
26	}	End of MainActivity class.

File: app/src/main/java/.../GraphicView.java
Purpose: Custom View that draws all graphical primitives on a Canvas using Paint.

Runtime Status: ACTIVE — All shapes on screen are drawn by this file.

Complete Source Code

```
 1| package com.example.drawsbasicgraphicalprimitives;
 2|
 3| import android.content.Context;
 4| import android.graphics.Canvas;
 5| import android.graphics.Color;
 6| import android.graphics.Paint;
 7| import android.view.View;
 8|
 9| import androidx.annotation.NonNull;
10|
11| public class GraphicView extends View {
12|     private final Paint paint;
13|
14|     public GraphicView(Context context) {
15|         super(context);
16|         paint = new Paint();
17|     }
18|
19|     @Override
20|     protected void onDraw(@NonNull Canvas canvas) {
21|         super.onDraw(canvas);
22|         paint.reset();
23|         paint.setAntiAlias(true);
24|
25|         // Draw background
26|         canvas.drawColor(Color.WHITE);
27|
28|         // Draw a circle
29|         paint.setColor(Color.RED);
30|         paint.setStrokeWidth(10);
31|         paint.setStyle(Paint.Style.STROKE);
32|         canvas.drawCircle(200, 200, 100, paint);
33|
34|         // Draw a rectangle
35|         paint.setColor(Color.GREEN);
36|         paint.setStyle(Paint.Style.FILL);
37|         canvas.drawRect(400, 100, 600, 300, paint);
38|
39|         // Draw a line
40|         paint.setColor(Color.BLUE);
41|         paint.setStrokeWidth(20);
42|         canvas.drawLine(100, 400, 700, 400, paint);
43|
44|         // Draw text
45|         paint.setColor(Color.BLACK);
46|         paint.setTextSize(60);
47|         paint.setStrokeWidth(2);
48|         canvas.drawText("Basic Graphical Primitives", 100, 600, paint);
49|
50|         // Draw an oval
51|         paint.setColor(Color.MAGENTA);
52|         paint.setStyle(Paint.Style.STROKE);
53|         canvas.drawOval(100, 700, 500, 900, paint);
54|
55|         // Draw an arc
56|         paint.setColor(Color.CYAN);
57|         paint.setStyle(Paint.Style.FILL);
58|         canvas.drawArc(600, 700, 900, 1000, 0, 90, true, paint);
59|
60|         // Draw points
61|         paint.setColor(Color.RED);
62|         paint.setStrokeWidth(30);
```

```
63|         canvas.drawPoint(800, 200, paint);
64|         canvas.drawPoint(850, 250, paint);
65|     }
66| }
```

Line-by-Line Explanation

Line	Code	Explanation
1	package ...	Package declaration.
3	import android.content.Context;	Context — provides access to app environment.
4	import android.graphics.Canvas;	Canvas — drawing surface for shapes.
5	import android.graphics.Color;	Color — predefined color constants (RED, GREEN, etc.).
6	import android.graphics.Paint;	Paint — holds style: color, stroke, fill mode, text size.
7	import android.view.View;	View — base class for all UI components.
9	import androidx.annotation.NonNull;	NonNull — annotation indicating parameter cannot be null.
11	public class GraphicView extends View {	Custom View class that handles all drawing.
12	private final Paint paint;	Paint object stored as class field; 'final' means reference never changes.
14	public GraphicView(Context context) {	Constructor — called when GraphicView is created.
15	super(context);	Calls View constructor with context.
16	paint = new Paint();	Creates new Paint object for drawing.
19	@Override	Overrides View's onDraw method.
20	protected void onDraw(@NonNull Canvas canvas) {	onDraw() — Android calls this to render the view.
21	super.onDraw(canvas);	Calls parent onDraw() first (good practice).
22	paint.reset();	Clears previous Paint settings before each draw.
23	paint.setAntiAlias(true);	Enables smooth edges on shapes (reduces jagged pixels).
26	canvas.drawColor(Color.WHITE);	Fills entire view background with white.
29	paint.setColor(Color.RED);	Sets drawing color to red for circle.
30	paint.setStrokeWidth(10);	Sets outline thickness to 10 pixels.
31	paint.setStyle(Paint.Style.STROKE);	STROKE = outline only (hollow circle).
32	canvas.drawCircle(200, 200, 100, paint);	Draws circle: center(200,200), radius 100.
35	paint.setColor(Color.GREEN);	Changes color to green for rectangle.
36	paint.setStyle(Paint.Style.FILL);	FILL = solid filled shape.
37	canvas.drawRect(400, 100, 600, 300, paint);	Draws rectangle: left=400, top=100, right=600, bottom=300.
40	paint.setColor(Color.BLUE);	Sets color to blue for line.
41	paint.setStrokeWidth(20);	Line thickness 20 pixels.
42	canvas.drawLine(100, 400, 700, 400, paint);	Draws line from (100,400) to (700,400).
45	paint.setColor(Color.BLACK);	Black color for text.
46	paint.setTextSize(60);	Text size 60 pixels.
47	paint.setStrokeWidth(2);	Resets stroke width for text rendering.
48	canvas.drawText("Basic Graphical Primitives", 100, 600, paint);	Draws text at position (100, 600).
51	paint.setColor(Color.MAGENTA);	Magenta color for oval.
52	paint.setStyle(Paint.Style.STROKE);	Outline-only oval.
53	canvas.drawOval(100, 700, 500, 900, paint);	Draws oval inside bounding box (100,700)-(500,900).
56	paint.setColor(Color.CYAN);	Cyan color for arc.
57	paint.setStyle(Paint.Style.FILL);	Filled arc (pie slice).
58	canvas.drawArc(600, 700, 900, 1000, 0, 90, true, paint);	Arc: bounding rect, start 0°, sweep 90°, useCenter=true.
61	paint.setColor(Color.RED);	Red color for points.
62	paint.setStrokeWidth(30);	Thick stroke makes points visible as dots.
63	canvas.drawPoint(800, 200, paint);	Draws a point at (800, 200).
64	canvas.drawPoint(850, 250, paint);	Draws second point at (850, 250).

File: app/src/main/java/.../FirstFragment.java
Purpose: First fragment with Next button. Demonstrates View Binding and Navigation Component.

Runtime Status: TEMPLATE — Not displayed in current app (MainActivity bypasses fragment navigation).

Complete Source Code

```
 1| package com.example.drawsbasicgraphicalprimitives;
 2|
 3| import android.os.Bundle;
 4| import android.view.LayoutInflater;
 5| import android.view.View;
 6| import android.view.ViewGroup;
 7|
 8| import androidx.annotation.NonNull;
 9| import androidx.fragment.app.Fragment;
10| import androidx.navigation.fragment.NavHostFragment;
11|
12| import com.example.drawsbasicgraphicalprimitives.databinding.FragmentFirstBinding;
13|
14| public class FirstFragment extends Fragment {
15|
16|     private FragmentFirstBinding binding;
17|
18|     @Override
19|     public View onCreateView(
20|             @NonNull LayoutInflater inflater, ViewGroup container,
21|             Bundle savedInstanceState
22|     ) {
23|
24|         binding = FragmentFirstBinding.inflate(inflater, container, false);
25|         return binding.getRoot();
26|
27|     }
28|
29|     public void onViewCreated(@NonNull View view, Bundle savedInstanceState) {
30|         super.onViewCreated(view, savedInstanceState);
31|
32|         binding.buttonFirst.setOnClickListener(v ->
33|                 NavHostFragment.findNavController(FirstFragment.this)
34|                         .navigate(R.id.action_FirstFragment_to_SecondFragment)
35|         );
36|     }
37|
38|     @Override
39|     public void onDestroyView() {
40|         super.onDestroyView();
41|         binding = null;
42|     }
43|
44| }
```

Line-by-Line Explanation

Line	Code	Explanation
1	package ...	Package declaration.
3	import android.os.Bundle;	Bundle for saving/restoring fragment state.
4	import android.view.LayoutInflater;	LayoutInflater — converts XML layout to View objects.
5	import android.view.View;	View base class.
6	import android.view.ViewGroup;	ViewGroup — container for child views.
8	import androidx.annotation.NonNull;	Non-null annotation.
9	import androidx.fragment.app.Fragment;	Fragment base class for reusable UI portions.
10	import ...NavHostFragment;	Navigation host for fragment-based navigation.
12	import ...FragmentFirstBinding;	Auto-generated View Binding class from fragment_first.xml.
14	public class FirstFragment extends Fragment {	First screen fragment (template).
16	private FragmentFirstBinding binding;	Binding object to access views without findViewById.
18	@Override	Overrides Fragment method.

19	public View onCreateView(...)	Called to create fragment's UI; returns root View.
24	binding = FragmentFirstBinding.inflate(...)	Inflates fragment_first.xml into binding object.
25	return binding.getRoot();	Returns root view of the inflated layout.
29	public void onViewCreated(...)	Called after view is created — setup click listeners here.
30	super.onViewCreated(...)	Calls parent onViewCreated.
32	binding.buttonFirst.setOnClickListener(v ->	Sets click listener on Next button from XML.
33	NavHostFragment.findNavController(...)	Gets navigation controller for this fragment.
34	.navigate(R.id.action_FirstFragment_to_SecondFragment)	Navigates to SecondFragment.
38	@Override	Overrides destroy method.
39	public void onDestroyView() {	Called when fragment view is destroyed.
41	binding = null;	Clears binding to prevent memory leaks.

File: app/src/main/java/.../SecondFragment.java
Purpose: Second fragment with Previous button. Navigates back to FirstFragment.

Runtime Status: TEMPLATE — Not displayed in current app.

Complete Source Code

```
 1| package com.example.drawsbasicgraphicalprimitives;
 2|
 3| import android.os.Bundle;
 4| import android.view.LayoutInflater;
 5| import android.view.View;
 6| import android.view.ViewGroup;
 7|
 8| import androidx.annotation.NonNull;
 9| import androidx.fragment.app.Fragment;
10| import androidx.navigation.fragment.NavHostFragment;
11|
12| import com.example.drawsbasicgraphicalprimitives.databinding.FragmentSecondBinding;
13|
14| public class SecondFragment extends Fragment {
15|
16|     private FragmentSecondBinding binding;
17|
18|     @Override
19|     public View onCreateView(
20|             @NonNull LayoutInflater inflater, ViewGroup container,
21|             Bundle savedInstanceState
22|     ) {
23|
24|         binding = FragmentSecondBinding.inflate(inflater, container, false);
25|         return binding.getRoot();
26|
27|     }
28|
29|     public void onViewCreated(@NonNull View view, Bundle savedInstanceState) {
30|         super.onViewCreated(view, savedInstanceState);
31|
32|         binding.buttonSecond.setOnClickListener(v ->
33|                 NavHostFragment.findNavController(SecondFragment.this)
34|                         .navigate(R.id.action_SecondFragment_to_FirstFragment)
35|         );
36|     }
37|
38|     @Override
39|     public void onDestroyView() {
40|         super.onDestroyView();
41|         binding = null;
42|     }
43|
44| }
```

Line-by-Line Explanation

Line	Code	Explanation
1	package ...	Package declaration.
12	import ...FragmentSecondBinding;	View Binding for fragment_second.xml.
14	public class SecondFragment extends Fragment {	Second screen fragment (template).
16	private FragmentSecondBinding binding;	Binding for second fragment views.
24	binding = FragmentSecondBinding.inflate(...)	Inflates fragment_second.xml.
32	binding.buttonSecond.setOnClickListener(v ->	Click listener on Previous button.
34	.navigate(R.id.action_SecondFragment_to_FirstFragment)	Navigates back to FirstFragment.
41	binding = null;	Prevents memory leak when view is destroyed.

5. XML Resource Files

File: app/src/main/AndroidManifest.xml

Purpose: Application blueprint. Android OS reads this before any Java code runs.

Runtime Status: ACTIVE — Required for app to launch.

Complete Source Code

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Drawsbasicgraphicalprimitives">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:theme="@style/Theme.Drawsbasicgraphicalprimitives">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Line-by-Line Explanation

Line	Code	Explanation
1	<?xml version="1.0" encoding="utf-8"?>	XML declaration — version and character encoding.
2	<manifest xmlns:android=...>	Root element; android namespace for Android attributes.
3	xmlns:tools=...	Tools namespace for design-time attributes.
5	<application>	Wraps all application-level components and settings.
6	android:allowBackup=true	Allows Android backup of app data.
7	android:dataExtractionRules=...	Points to cloud backup rules (Android 12+).
8	android:fullBackupContent=...	Points to full backup rules file.
9	android:icon=@mipmap/ic_launcher	App icon shown in launcher.
10	android:label=@string/app_name	App name from strings.xml.
11	android:roundIcon=...	Round icon for supported launchers.
12	android:supportsRtl=true	Supports right-to-left languages.
13	android:theme=...	Default theme for entire application.
14	<activity>	Declares an Activity component.
15	android:name=.MainActivity	Short name for MainActivity (package prefix added automatically).
16	android:exported=true	Allows external apps/system to launch this activity.
17	android:theme=...	Theme applied to this activity.
18	<intent-filter>	Declares how other components can start this activity.
19	action MAIN	Indicates this is an entry point of the application.
21	category LAUNCHER	Makes this activity appear in the app drawer.
22	</intent-filter>	End of intent filter.
23	</activity>	End of MainActivity declaration.
24	</application>	End of application block.
26	</manifest>	End of manifest file.

File: app/src/main/res/layout/activity_main.xml
Purpose: Main screen shell with Toolbar, content area, and Floating Action Button.

Runtime Status: TEMPLATE — Not used because MainActivity sets GraphicView directly.

Complete Source Code

```
 1| <?xml version="1.0" encoding="utf-8"?>
 2| <androidx.coordinatorlayout.widget.CoordinatorLayout xmlns:android="http://schemas.android.com/apk/res/android"
 3|     xmlns:app="http://schemas.android.com/apk/res-auto"
 4|     xmlns:tools="http://schemas.android.com/tools"
 5|     android:id="@+id/main"
 6|     android:layout_width="match_parent"
 7|     android:layout_height="match_parent"
 8|     android:fitsSystemWindows="true"
 9|     tools:context=".MainActivity">
10|
11|     <com.google.android.material.appbar.AppBarLayout
12|         android:layout_width="match_parent"
13|         android:layout_height="wrap_content"
14|         android:fitsSystemWindows="true">
15|
16|         <com.google.android.material.appbar.MaterialToolbar
17|             android:id="@+id/toolbar"
18|             android:layout_width="match_parent"
19|             android:layout_height="?attr/actionBarSize" />
20|
21|     </com.google.android.material.appbar.AppBarLayout>
22|
23|     <include layout="@layout/content_main" />
24|
25|     <com.google.android.material.floatingactionbutton.FloatingActionButton
26|         android:id="@+id/fab"
27|         android:layout_width="wrap_content"
28|         android:layout_height="wrap_content"
29|         android:layout_gravity="bottom|end"
30|         android:layout_marginEnd="@dimen/fab_margin"
31|         android:layout_marginBottom="16dp"
32|         app:srcCompat="@android:drawable/ic_dialog_email" />
33|
34| </androidx.coordinatorlayout.widget.CoordinatorLayout>
```

Line-by-Line Explanation

Line	Code	Explanation
1	<?xml version="1.0"...	XML header.
2	CoordinatorLayout	Root layout that coordinates child views (Material Design).
5	android:id=@+id/main	Creates resource ID 'main' for this layout.
6	layout_width=match_parent	Width fills entire screen width.
7	layout_height=match_parent	Height fills entire screen height.
8	fitsSystemWindows=true	Adjusts for system windows (status bar).
9	tools:context=.MainActivity	Design-time hint linking layout to MainActivity.
11	AppBarLayout	Container for app bar / toolbar at top.
16	MaterialToolbar	Material Design toolbar (title bar).
17	android:id=@+id/toolbar	ID for accessing toolbar in Java.
19	layout_height=?attr/actionBarSize	Height matches theme's action bar size.
23	<include layout=@layout/content_main />	Embeds content_main.xml inside this layout.
25	FloatingActionButton	Circular button floating at bottom-right.
26	android:id=@+id/fab	ID for FAB.
29	layout_gravity=bottom|end	Positions FAB at bottom-right corner.
30	layout_marginEnd=@dimen/fab_margin	Right margin from dimens.xml (16dp).
32	srcCompat=ic_dialog_email	Email icon on the FAB.

File: app/src/main/res/layout/content_main.xml
Purpose: Hosts NavHostFragment for fragment-based screen navigation.

Runtime Status: TEMPLATE — Not used in current app.

Complete Source Code

```
1| <?xml version="1.0" encoding="utf-8"?>
2| <androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
3|     xmlns:app="http://schemas.android.com/apk/res-auto"
4|     android:layout_width="match_parent"
5|     android:layout_height="match_parent"
6|     app:layout_behavior="@string/appbar_scrolling_view_behavior">
7|
8|     <androidx.fragment.app.FragmentContainerView
9|         android:id="@+id/nav_host_fragment_content_main"
10|         android:name="androidx.navigation.fragment.NavHostFragment"
11|         android:layout_width="0dp"
12|         android:layout_height="0dp"
13|         app:defaultNavHost="true"
14|         app:layout_constraintBottom_toBottomOf="parent"
15|         app:layout_constraintEnd_toEndOf="parent"
16|         app:layout_constraintStart_toStartOf="parent"
17|         app:layout_constraintTop_toTopOf="parent"
18|         app:navGraph="@navigation/nav_graph" />
19| </androidx.constraintlayout.widget.ConstraintLayout>
```

Line-by-Line Explanation

Line	Code	Explanation
2	ConstraintLayout	Flexible layout using constraints between views.
6	layout_behavior=appbar_scrolling_view_behavior	Allows content to scroll under app bar.
8	FragmentContainerView	Container that hosts fragments for navigation.
9	android:id=nav_host_fragment_content_main	ID of navigation host container.
10	android:name=NavHostFragment	Uses NavHostFragment class to manage navigation.
11	layout_width=0dp	0dp with constraints = match constraint (fill available space).
13	defaultNavHost=true	Handles system back button for fragment navigation.
14	layout_constraintBottom_toBottomOf=parent	Bottom edge aligned to parent bottom.
18	navGraph=@navigation/nav_graph	Links to navigation graph defining fragment routes.

File: app/src/main/res/layout/fragment_first.xml

Purpose: UI layout for First Fragment.

Runtime Status: TEMPLATE — Not displayed in current app.

Complete Source Code

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.core.widget.NestedScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".FirstFragment">

    <androidx.constraintlayout.widget.ConstraintLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:padding="16dp">

        <Button
            android:id="@+id/button_first"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/next"
            app:layout_constraintBottom_toTopOf="@id/buttonFirst"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent" />

        <TextView
            android:id="@+id/buttonFirst"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp"
            android:text="@string/lorem_ipsum"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@id/button_first" />
    </androidx.constraintlayout.widget.ConstraintLayout>
</androidx.core.widget.NestedScrollView>
```

Line-by-Line Explanation

Line	Code	Explanation
2	NestedScrollView	Scrollable container for long content.
9	ConstraintLayout	Inner layout with constraint-based positioning.
12	android:padding=16dp	16 density-independent pixels padding on all sides.
14	Button	Next button — ID links to binding.next in Java.
15	android:id=@+id/button_first	Resource ID referenced in Fragment Java as binding.buttonFirst.
18	android:text=@string/...	Button label from strings.xml (Next).
19	layout_constraintBottom_toTopOf=...	Button bottom constrained above TextView.
24	TextView	Displays lorem ipsum placeholder text.
29	android:text=@string/lorem_ipsum	Long placeholder text from strings.xml.
33	layout_constraintTop_toBottomOf=...	TextView placed below the button.

File: app/src/main/res/layout/fragment_second.xml

Purpose: UI layout for Second Fragment.

Runtime Status: TEMPLATE — Not displayed in current app.

Complete Source Code

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.core.widget.NestedScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".SecondFragment">

    <androidx.constraintlayout.widget.ConstraintLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:padding="16dp">

        <Button
            android:id="@+id/button_second"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/previous"
            app:layout_constraintBottom_toTopOf="@id/buttonSecond"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent" />

        <TextView
            android:id="@+id/buttonSecond"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp"
            android:text="@string/lorem_ipsum"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@id/button_second" />
    </androidx.constraintlayout.widget.ConstraintLayout>
</androidx.core.widget.NestedScrollView>
```

Line-by-Line Explanation

Line	Code	Explanation
2	NestedScrollView	Scrollable container for long content.
9	ConstraintLayout	Inner layout with constraint-based positioning.
12	android:padding=16dp	16 density-independent pixels padding on all sides.
14	Button	Previous button — ID links to binding.buttonSecond in Java.
15	android:id=@+id/button_second	Resource ID referenced in Fragment Java as binding.buttonSecond.
18	android:text=@string/...	Button label from strings.xml (Previous).
19	layout_constraintBottom_toTopOf=...	Button bottom constrained above TextView.
24	TextView	Displays lorem ipsum placeholder text.
29	android:text=@string/lorem_ipsum	Long placeholder text from strings.xml.
33	layout_constraintTop_toBottomOf=...	TextView placed below the button.

File: app/src/main/res/navigation/nav_graph.xml

Purpose: Defines fragment destinations and navigation actions between them.

Runtime Status: TEMPLATE — Not active in current app.

Complete Source Code

```
 1| <?xml version="1.0" encoding="utf-8"?>
 2| <navigation xmlns:android="http://schemas.android.com/apk/res/android"
 3|     xmlns:app="http://schemas.android.com/apk/res-auto"
 4|     xmlns:tools="http://schemas.android.com/tools"
 5|     android:id="@+id/nav_graph"
 6|     app:startDestination="@id/FirstFragment">
 7|
 8|     <fragment
 9|         android:id="@+id/FirstFragment"
10|         android:name="com.example.drawsbasicgraphicalprimitives.FirstFragment"
11|         android:label="@string/first_fragment_label"
12|         tools:layout="@layout/fragment_first">
13|
14|         <action
15|             android:id="@+id/action_FirstFragment_to_SecondFragment"
16|             app:destination="@id/SecondFragment" />
17|     </fragment>
18|     <fragment
19|         android:id="@+id/SecondFragment"
20|         android:name="com.example.drawsbasicgraphicalprimitives.SecondFragment"
21|         android:label="@string/second_fragment_label"
22|         tools:layout="@layout/fragment_second">
23|
24|         <action
25|             android:id="@+id/action_SecondFragment_to_FirstFragment"
26|             app:destination="@id/FirstFragment" />
27|     </fragment>
28| </navigation>
```

Line-by-Line Explanation

Line	Code	Explanation
2	<navigation>	Root of Navigation Component graph.
5	android:id=nav_graph	Unique ID for this navigation graph.
6	startDestination=FirstFragment	First screen shown when navigation starts.
8	<fragment>	Declares a navigable destination.
9	android:id=FirstFragment	Destination ID used in navigate() calls.
10	android:name=...FirstFragment	Fully qualified Java class for this fragment.
11	android:label=...	Title shown in action bar for this destination.
12	tools:layout=fragment_first	Design-time preview layout.
14	<action>	Defines a navigation transition.
15	action_FirstFragment_to_SecondFragment	Action ID used in R.id.action_... in Java.
16	destination=SecondFragment	Target destination when action is triggered.
18	SecondFragment block	Second destination with reverse navigation action.

File: app/src/main/res/menu/menu_main.xml

Purpose: Options menu with Settings item.

Runtime Status: TEMPLATE — Not wired up in current MainActivity.

Complete Source Code

```
1| <menu xmlns:android="http://schemas.android.com/apk/res/android"
2|     xmlns:app="http://schemas.android.com/apk/res-auto"
3|     xmlns:tools="http://schemas.android.com/tools"
4|     tools:context="com.example.drawsbasicgraphicalprimitives.MainActivity">
5|     <item
6|         android:id="@+id/action_settings"
7|         android:orderInCategory="100"
8|         android:title="@string/action_settings"
9|         app:showAsAction="never" />
10| </menu>
```

Line-by-Line Explanation

Line	Code	Explanation
1	<menu>	Root element for options menu.
4	tools:context=MainActivity	Design-time link to MainActivity.
5	<item>	Single menu item definition.
6	android:id=action_settings	ID for handling menu click in Java.
7	orderInCategory=100	Sort order within menu category.
8	title=@string/action_settings	Menu item text: 'Settings'.
9	showAsAction=never	Always show in overflow menu (three dots), not action bar.

File: app/src/main/res/values/strings.xml

Purpose: Centralized text strings referenced as @string/name throughout XML and Java.

Runtime Status: ACTIVE — app_name used in manifest.

Complete Source Code

```
1| <resources>
2|     <string name="app_name">Drawsbasicgraphicalprimitives</string>
3|     <string name="action_settings">Settings</string>
4|     <string name="first_fragment_label">First Fragment</string>
5|     <string name="second_fragment_label">Second Fragment</string>
6|     <string name="next">Next</string>
7|     <string name="previous">Previous</string>
8|     <string name="lorem_ipsum">Lorem ipsum dolor sit amet... (long placeholder text)</string>
9| </resources>
```

Line-by-Line Explanation

Line	Code	Explanation
1	<resources>	Root container for all string resources.
2	app_name	Application name shown in launcher and title.
3	action_settings	Text for Settings menu item.
4	first_fragment_label	Title for First Fragment in navigation.
5	second_fragment_label	Title for Second Fragment.
6	next	Label on Next button in First Fragment.
7	previous	Label on Previous button in Second Fragment.
8	lorem_ipsum	Long placeholder paragraph for fragment TextViews.

File: app/src/main/res/values/colors.xml

Purpose: Named color resources. Referenced as @color/black or @color/white.

Runtime Status: ACTIVE — available for themes and layouts.

Complete Source Code

```
1| <?xml version="1.0" encoding="utf-8"?>
2| <resources>
3|     <color name="black">#FF000000</color>
4|     <color name="white">#FFFFFFFF</color>
5| </resources>
```

Line-by-Line Explanation

Line	Code	Explanation
3	black #FF000000	Fully opaque black (ARGB: Alpha FF, RGB 000000).
4	white #FFFFFFFF	Fully opaque white.

File: app/src/main/res/values/dimens.xml

Purpose: Dimension values used in layouts.

Runtime Status: ACTIVE — referenced by activity_main.xml FAB margin.

Complete Source Code

```
1| <resources>
2|     <dimen name="fab_margin">16dp</dimen>
3| </resources>
```

Line-by-Line Explanation

Line	Code	Explanation
2	fab_margin 16dp	16 density-independent pixels margin for FAB in activity_main.xml.

File: app/src/main/res/values/themes.xml

Purpose: Defines application visual theme (Material 3).

Runtime Status: ACTIVE — applied via manifest.

Complete Source Code

```
1 | <resources xmlns:tools="http://schemas.android.com/tools">
2 |     <style name="Base.Theme.Drawsbasicgraphicalprimitives" parent="Theme.Material3.DayNight.NoActionBar">
3 |     </style>
4 |     <style name="Theme.Drawsbasicgraphicalprimitives" parent="Base.Theme.Drawsbasicgraphicalprimitives" />
5 | </resources>
```

Line-by-Line Explanation

Line	Code	Explanation
2	Base.Theme...	Base theme extending Material 3 DayNight without action bar.
4	Theme.Drawsbasicgraphicalprimitives	App theme name referenced in AndroidManifest.xml.

File: app/src/main/res/values-v23/themes.xml

Purpose: Theme overrides for API level 23+ (Android 6.0+) enabling edge-to-edge display.

Runtime Status: ACTIVE — supports EdgeToEdge.enable() in MainActivity.

Complete Source Code

```
1 | <resources xmlns:tools="http://schemas.android.com/tools">
2 |     <style name="Theme.Drawsbasicgraphicalprimitives" parent="Base.Theme.Drawsbasicgraphicalprimitives">
3 |         <item name="android:navigationBarColor">@android:color/transparent</item>
4 |         <item name="android:statusBarColor">@android:color/transparent</item>
5 |         <item name="android:windowLightStatusBar">?attr/isLightTheme</item>
6 |     </style>
7 | </resources>
```

Line-by-Line Explanation

Line	Code	Explanation
3	navigationBarColor transparent	Makes navigation bar transparent for edge-to-edge.
4	statusBarColor transparent	Makes status bar transparent.
5	windowLightStatusBar	Uses dark icons on light status bar when theme is light.

6. Build Configuration

```
android {
    namespace = "com.example.drawsbasicgraphicalprimitives"
    defaultConfig {
        applicationId = "com.example.drawsbasicgraphicalprimitives"
        minSdk = 26
        targetSdk = 36
    }
    buildFeatures {
        viewBinding = true
    }
}
```

minSdk 26: App requires Android 8.0 or higher.

viewBinding true: Auto-generates binding classes for XML layouts (used by fragments).

7. Student Exercises

1. Trace the launch path: Manifest → MainActivity → GraphicView → onDraw().

2. Change circle color from RED to YELLOW in GraphicView.java and run the app.

3. Add a new shape (e.g., a filled triangle) in onDraw().

4. Explain why activity_main.xml is not displayed when the app runs.

5. Map button_first in fragment_first.xml to binding.buttonFirst in FirstFragment.java.

6. Add a new color in colors.xml and use it in GraphicView for text color.

8. Viva Questions

• What is the difference between Activity and View?

• When is onDraw() called?

• What is the purpose of AndroidManifest.xml?

• Explain Paint.Style.STROKE vs FILL.

• What is View Binding?

• Which files are active at runtime vs template-only?